

Operating Systems Project

Phase 3 Report

Group Members:

Yixuan Ma ym3116

Uditi Sharma us2133

1 Architecture and Design

- **Main thread is a pure dispatcher:** it owns the listening socket, calls `accept()` in a loop, and immediately hands each new connection off to a worker thread. It never blocks on client I/O.
- **Worker threads are detached POSIX threads, each running `handle_client()`.** A thread owns its client socket for the full session — receive command, parse, execute, send output — and close it on disconnect.
- **Process isolation per command:** every command runs in a `fork()`ed child whose `stdout/stderr` are redirected into a pipe. This guarantees one client's `cd`, signals, or exit from a builtin cannot disturb the server or other threads.
- **Shared state is minimal and explicitly protected:** a global client counter and all `printf` log lines are the only cross-thread resources, and each is guarded by a dedicated mutex.

Key design decisions:

The server uses one detached thread per client rather than a thread pool or `select()`-based multiplexing: it maps naturally to the "each thread handles one client" requirement and keeps the per-client logic a straightforward top-to-bottom function.

Each thread then `fork()`s a child for every command and captures its output through a pipe — this isolates the Phase 1 shell engine (which freely uses `chdir`, `dup2`, and `exit`) from the server process, so one client's command can never corrupt another's state or crash the server.

Finally, shared state is deliberately minimized to two variables — a global client counter and `stdout` — each guarded by its own mutex (`counter_mutex`, `print_mutex`); splitting them keeps critical sections tiny so a thread writing a log line never blocks another thread from obtaining a client number.

2 Implementation Highlights

1. Per-client thread dispatch

`main()` accepts a connection, assigns a unique client number under `counter_mutex`, packages the socket/IP/port/number into a heap-allocated `ClientArgs`, and spawns a detached thread:

```

pthread_mutex_lock(&counter_mutex);
int client_num = ++client_counter;
pthread_mutex_unlock(&counter_mutex);

ClientArgs *args = malloc(sizeof(ClientArgs));
args->client_fd    = client_fd;
args->client_num   = client_num;
args->client_port  = ntohs(client_addr.sin_port);
strncpy(args->client_ip, inet_ntoa(client_addr.sin_addr),
INET_ADDRSTRLEN - 1);
pthread_create(&tid, NULL, client_thread, args);
pthread_detach(tid);

```

Detaching avoids the need for `pthread_join`, so the server never accumulates zombie thread records.

2. Per-command execution via `fork()` + `pipe()`

Each command received by a thread is executed in a child process whose output is captured through a pipe:

```

int pipe_fds[2]; pipe(pipe_fds);
pid_t pid = fork();
if (pid == 0) {                                /* child */
    dup2(pipe_fds[1], STDOUT_FILENO);
    dup2(pipe_fds[1], STDERR_FILENO);
    execute_pipeline(pl);
    exit(0);
}
/* parent reads in READ_CHUNK increments, growing buf up to
MAX_OUTPUT (1 MB) */

```

The parent reads in 4 KB chunks and grows the buffer geometrically up to a 1 MB cap, preventing unbounded memory use on yes-like commands.

3. Thread-safe logging and client numbering

Two mutexes guard the only shared state:

```
static pthread_mutex_t counter_mutex = PTHREAD_MUTEX_INITIALIZER;  
static pthread_mutex_t print_mutex  = PTHREAD_MUTEX_INITIALIZER;
```

Every log line is wrapped in `pthread_mutex_lock(&print_mutex); ... fflush(stdout); pthread_mutex_unlock(&print_mutex);` so concurrent threads never interleave characters in the server terminal. Log lines follow the required `[TAG] [Client #N - IP:port] ...` format.

4. Length-prefix message protocol

Every server → client response is a 4-byte network-order length followed by that many bytes of output. `send_all()` loops on `send()` to handle partial writes, and the client mirrors `recv_all()` loops on `recv()` to tolerate TCP segmentation.

```
uint32_t net_len = htonl((uint32_t)out_len);  
send_all(client_fd, &net_len, sizeof(net_len));  
send_all(client_fd, output, out_len);
```

3 Execution Instructions

1. `cd myshell-os-project`
2. Run : `make` # builds `myshell`, `myshell_server`, `myshell_client`
3. Run this in one terminal: `./myshell_server <port>` (example: `./myshell_server 9090`)
4. Run this in another terminal: `./myshell_client <server_host> <port>` (example: `./myshell_client 127.0.0.1 9090`)
5. After finishing everything run: `make clean` # removes all object files and binaries

Testing

All tests were run locally on macOS (Darwin 23.3.0) with the server and client compiled via **make**. The server was started on **127.0.0.1** and clients connected over loopback TCP.

Tests are split into two groups:

- **Phase 3 tests** (Tests 1–3, 6–7): verify multithreaded execution, correct server log format, concurrent client handling, and graceful disconnect.
 - **Cumulative verification** (Tests 4–5): confirm that Phase 1 features (pipelines, output redirection) still work correctly when commands are routed through the multithreaded server, as required by the assignment's cumulative policy.
-

Test 1 — Single Client: Basic Commands (**pwd**, **ls**, **echo**)

Purpose: Verify that a single connected client can execute standard commands and receive correct output.

Commands sent by client:

```
$ pwd
```

```
$ ls
```

```
$ echo hello world
```

```
$ exit
```

Server output:

```
[INFO] Client #1 connected from 127.0.0.1:53595. Assigned to Thread-1.
```

```
[RECEIVED] [Client #1 - 127.0.0.1:53595] Received command: "pwd"
```

```
[EXECUTING] [Client #1 - 127.0.0.1:53595] Executing command: "pwd"
```

```
[OUTPUT] [Client #1 - 127.0.0.1:53595] Sending output to client:
```

```
/path/to/project
```

```
[RECEIVED] [Client #1 - 127.0.0.1:53595] Received command: "ls"
```

```
[EXECUTING] [Client #1 - 127.0.0.1:53595] Executing command: "ls"
```

[OUTPUT] [Client #1 - 127.0.0.1:53595] Sending output to client:

Makefile include src ...

[RECEIVED] [Client #1 - 127.0.0.1:53595] Received command: "echo hello world"

[EXECUTING] [Client #1 - 127.0.0.1:53595] Executing command: "echo hello world"

[OUTPUT] [Client #1 - 127.0.0.1:53595] Sending output to client:

hello world

[RECEIVED] [Client #1 - 127.0.0.1:53595] Received command: "exit"

[INFO] [Client #1 - 127.0.0.1:53595] Client requested disconnect. Closing connection.

[INFO] Client #1 disconnected.

Client output:

\$ pwd

/path/to/project

\$ ls

Makefile include src ...

\$ echo hello world

hello world

\$ exit

[Client] Disconnected from server.

Result: PASS — output matches required format; client receives correct results.

Test 2 — Unknown Command (Error Handling)

Purpose: Verify that an unrecognised command produces the correct [ERROR] and [OUTPUT] log entries and sends an error message back to the client.

Command sent:

```
$ unknowncmd
```

Server output:

```
[RECEIVED] [Client #2 - 127.0.0.1:53596] Received command: "unknowncmd"
```

```
[EXECUTING] [Client #2 - 127.0.0.1:53596] Executing command: "unknowncmd"
```

```
[ERROR] [Client #2 - 127.0.0.1:53596] Command not found: "unknowncmd"
```

```
[OUTPUT] [Client #2 - 127.0.0.1:53596] Sending error message to client: "Command not found."
```

Client output:

```
$ unknowncmd
```

```
Command not found.
```

Result: PASS — error is correctly detected, logged with [ERROR] tag, and sent to the client.

Test 3 — Three Concurrent Clients

Purpose: Verify that multiple clients are served simultaneously without blocking each other, and that each log line correctly identifies its client by number, IP, and port.

Setup: Three clients connected at the same time, each sending a different command:

- Client #5: `pwd` then `exit`
- Client #6: `unknowncmd` then `exit`
- Client #7: `ls src | grep .c` then `exit`

Server output (actual, captured during test):

[INFO] Client #5 connected from 127.0.0.1:53599. Assigned to Thread-5.

[RECEIVED] [Client #5 - 127.0.0.1:53599] Received command: "pwd"

[EXECUTING] [Client #5 - 127.0.0.1:53599] Executing command: "pwd"

[INFO] Client #6 connected from 127.0.0.1:53600. Assigned to Thread-6.

[RECEIVED] [Client #6 - 127.0.0.1:53600] Received command: "unknowncmd"

[EXECUTING] [Client #6 - 127.0.0.1:53600] Executing command: "unknowncmd"

[INFO] Client #7 connected from 127.0.0.1:53601. Assigned to Thread-7.

[RECEIVED] [Client #7 - 127.0.0.1:53601] Received command: "ls src | grep .c"

[EXECUTING] [Client #7 - 127.0.0.1:53601] Executing command: "ls src | grep .c"

[OUTPUT] [Client #5 - 127.0.0.1:53599] Sending output to client:

/path/to/project

[INFO] [Client #5 - ...] Client requested disconnect. Closing connection.

[INFO] Client #5 disconnected.

[ERROR] [Client #6 - 127.0.0.1:53600] Command not found: "unknowncmd"

[OUTPUT] [Client #6 - ...] Sending error message to client: "Command not found."

[INFO] Client #6 disconnected.

[OUTPUT] [Client #7 - 127.0.0.1:53601] Sending output to client:

client.c exec.c main.c parser.c pipe.c redir.c server.c

[INFO] Client #7 disconnected.

Result: PASS — all three clients handled concurrently; mutex-protected log lines do not interleave; each line correctly labels its client.

Test 4 — Cumulative Verification: Phase 1 Pipeline (cmd1 | cmd2)

Purpose: Confirm that Phase 1 pipeline functionality is preserved when commands are executed through the Phase 3 multithreaded server. The assignment states all prior-phase functionality must remain correct.

Command sent:

```
$ ls src | grep .c
```

Client output:

```
client.c
```

```
exec.c
```

```
main.c
```

```
parser.c
```

```
pipe.c
```

```
redir.c
```

```
server.c
```

Result: PASS — pipeline correctly pipes `ls src` into `grep .c`; output received by client is correct.

Test 5 — Cumulative Verification: Phase 1 Output Redirection (>)

Purpose: Confirm that Phase 1 output redirection is preserved through the Phase 3 server. A command that redirects to a file should produce a zero-length socket response, which the server and client must both handle without error.

Commands sent:

```
$ echo phase3test > /tmp/test_out.txt
```

```
$ cat /tmp/test_out.txt
```

Server output for the redirect command:

```
[OUTPUT] [Client #4 - ...] Sending output to client:
```

(no output — may have been redirected to a file)

Client output for **cat**:

```
phase3test
```

Result: PASS — redirect silently writes to file; **cat** confirms the file was written correctly; zero-length response handled without error.

Test 6 — EOF / Ctrl-D Disconnect

Purpose: Verify that when a client's stdin closes (EOF / Ctrl-D) rather than typing **exit**, the server cleans up the connection gracefully without hanging or crashing.

Scenario: Client sends **pwd** then closes stdin.

Expected behaviour: **client.c** sends the string "**exit**" on EOF, so the server receives it and logs a normal disconnect. Server continues accepting new clients afterwards.

Result: PASS — connection closed cleanly; server printed disconnect messages and remained running.

Test 7 — Server Stability Across Multiple Sessions

Purpose: Verify the server keeps accepting new clients after one disconnects and that client numbering increments correctly across all sessions.

Scenario: Seven clients connected sequentially and in parallel across all tests above.

Result: PASS — client numbers incremented from #1 through #7 without collision; server remained stable throughout.

Summary of Test Results

#	Test	Type	Result
1	Basic commands (<code>pwd</code> , <code>ls</code> , <code>echo</code>)	Phase 3	PASS
2	Unknown command error handling	Phase 3	PASS
3	Three concurrent clients	Phase 3	PASS
4	Pipeline (<code>ls src \</code> <code>grep .c</code>)	Cumulative (Phase 1)	PASS
5	Output redirection (<code>></code>)	Cumulative (Phase 1)	PASS
6	EOF / Ctrl-D disconnect	Phase 3	PASS
7	Server stability across sessions	Phase 3	PASS

Challenges

1. Thread Safety on Shared State

The biggest challenge in Phase 3 was ensuring correctness when multiple threads access shared variables simultaneously. Two pieces of shared state required protection:

- **client_counter** — incremented each time a new connection is accepted. Without a lock, two threads racing through `accept()` at the same time could read the same value and assign duplicate client numbers. This was solved by wrapping the increment in `counter_mutex`.
- **printf / fflush calls** — without synchronisation, log lines from two concurrent threads interleave mid-line on stdout, producing unreadable output. A dedicated `print_mutex` serialises every print block across all threads, so each log line is always written atomically.

2. Passing Connection Data to a Thread Safely

A subtle but critical bug would arise if `ClientArgs` were allocated on the stack in `main()`. Because `main()` loops back to `accept()` immediately after `pthread_create()`, the stack frame containing the struct gets reused before the new thread has a chance to read its fields. The fix was to heap-allocate `ClientArgs` with `malloc()` for each connection and free it inside the thread once it is no longer needed.

3. Updating the Log Format from Phase 2 to Phase 3

The Phase 2 server logged generic messages like "[INFO] Client connected." with no client identity information. Updating every log statement to include [Client #N - IP:port] required identifying all the places where `client_ip` and `client_port` were needed but had been suppressed with `(void)` casts as placeholders. Once the variables were properly referenced in each `printf`, the suppressor lines were removed.

4. Capturing Output from the Shell Engine Per-Client

The Phase 1 `execute_pipeline()` function writes to `stdout` directly. To capture its output and send it over the socket, the server forks a child process, redirects the child's `stdout` and `stderr` to a pipe using `dup2()`, and reads from the pipe in the parent. The challenge was correctly sizing the output buffer — a growable `realloc` loop was used to handle commands that produce large amounts of output without hitting a fixed limit.

5. Handling Partial Sends and Receives

TCP does not guarantee that a single `send()` or `recv()` transfers all requested bytes in one call. For the 4-byte length header and for large command outputs this could silently truncate data. The `send_all()` and `recv_all()` wrapper functions loop until all bytes are transferred (or the connection is lost), ensuring the length-prefix protocol works correctly for all output sizes.

Division of Tasks

Task	Team Member
Threading implementation — <code>pthread_create</code> , <code>pthread_detach</code> , <code>client_thread</code> entry point	Annie
Mutex design — <code>counter_mutex</code> for client numbering, <code>print_mutex</code> for serialised logging	Annie
Architecture and Design section of report	Annie
Execution Instructions section of report	Annie
Updating server log format to Phase 3 specification (<code>[Client #N - IP:port]</code>)	Uditi
Removing <code>(void)client_ip / (void)client_port</code> suppressor lines	Uditi
Overall testing — single client, concurrent clients, pipelines, redirection, EOF disconnect	Uditi
Testing section of report	Uditi
Challenges section of report	Uditi
References section of report	Uditi

References

1. **POSIX Threads (pthreads) Programming** — Open Group Base Specifications Issue 7, IEEE Std 1003.1-2017. <https://pubs.opengroup.org/onlinepubs/9699919799/>
2. **Linux man pages: `pthread_create(3)`, `pthread_mutex_lock(3)`, `pthread_detach(3)`**. Available via `man pthread_create` on any POSIX system.
3. **Beej's Guide to Network Programming** — Brian "Beej" Hall. Comprehensive reference for the BSD socket API (`socket`, `bind`, `listen`, `accept`, `recv`, `send`). <https://beej.us/guide/bgnet/>
4. **`recv(2)`, `send(2)`, `accept(2)`** — **Linux Programmer's Manual**. Used to understand partial send/recv behaviour and the need for `send_all` / `recv_all` wrappers.
5. **`inet_ntoa(3)`, `getaddrinfo(3)`** — **Linux Programmer's Manual**. Used for IP address conversion between binary and human-readable text form.
6. **W. Richard Stevens, Bill Fenner, Andrew M. Rudoff — "UNIX Network Programming, Volume 1: The Sockets Networking API", 3rd Edition**. Prentice Hall, 2003. Reference for TCP socket programming patterns and concurrent server design using threads.
7. **`htonl(3)` / `ntohl(3)` / `htons(3)`** — **Network byte order conversion, Linux Programmer's Manual**. Used for the 4-byte length-prefix protocol to ensure correct byte ordering across machines.